GroundLoop Initial Technical Design and Research Blueprint

VERSIONED NEURAL OBSERVATIONS / FACTORIZED IVM / RISK-BOUNDED REFRESH

Initial architecture, formal maintained views, delta rules, consistency protocol, research extensions, and implementation plan

GroundLoop FYP | Design candidate v0.1 | 17 July 2026

Version: 0.1 design candidate Date: 17 July 2026 Status: design for review; implementation should begin only after the core decisions in Section 22 are accepted or amended

Executive verdict

The initial GroundLoop direction is worth pursuing, but the earlier design is not yet precise enough to implement. Its strongest idea is the boundary between uncertain semantic computation and exact relational maintenance:

```
documents and generated answers
        -> versioned neural observations
        -> exact incremental view maintenance
        -> claim and answer grounding state
```

That boundary gives GroundLoop a real AI component and a real database component. It avoids the false premise that raw language, embeddings, or LLM outputs are deterministic relations. It also gives the project an exact systems claim that can be tested independently of model quality.

The first design nevertheless has six important defects:

1. A stored SUPPORT label conflates model output with a thresholding policy.
2. Passage identity is ambiguous after replacement or rechunking.
3. Deletion and insertion are described too symmetrically even though only

deletion has an exact reverse-dependency path.

4. Long-running neural work has no sufficiently precise consistency model.
5. A naive join of answers, claims, passages, observations, requirements, and

groups can materialize a large and mostly redundant intermediate result.

6. REGENERATION_RECOMMENDED is mixed with grounding facts even though it is

an action policy, not an evidence state.

This document corrects those defects. It recommends a versioned semantic delta architecture with four defining mechanisms:

- immutable neural score observations plus incrementally maintained policy

decisions;

- a bounded, factorized evidence view tree maintained by signed deltas;
- asymmetric bidirectional impact discovery: exact for withdrawal and

approximate for admission of new evidence;

- epoch-level publication with explicit pending-work bounds.

The most promising research extension is risk-bounded neural IVM: the exact relational engine maintains consequences of completed observations, while a budgeted scheduler selects the next expensive semantic deltas to acquire by expected reduction in stale-answer risk. This is not yet established novelty. It is a concrete, falsifiable design hypothesis that must be compared with the closest work before any novelty claim.

1. Problem definition

Let a corpus evolve through epochs e = 0, 1, 2, ... At each epoch, a set of previously generated answers is registered. An answer contains atomic claims. Claims are grounded by direct evidence or by bounded groups of evidence requirements. A document insertion, deletion, or replacement can change which passage versions are active and therefore which stored evidence judgments are applicable.

GroundLoop must maintain, for every registered claim and answer:

- the active supporting and refuting observations;
- complete alternative evidence groups;
- the current canonical grounding state;
- a compact explanation of any state transition;
- whether semantic evaluation for the latest corpus epoch is complete;
- the model calls, latency, and state avoided relative to full refresh.

The primary research question is:

> *How can a RAG system maintain claim-level grounding for previously generated answers over an evolving document collection while using substantially fewer expensive semantic evaluations than full retrieval and verification?*

The exact claim is deliberately narrower:

> *For the same active versions, stored neural score observations, decision policy, and bounded evidence structure, GroundLoop's incremental state equals full evaluation of the declared relational semantics.*

The empirical claims concern candidate discovery, verifier quality, risk, latency, and neural-call savings. GroundLoop does not maintain objective truth.

2. Why this is not merely a RAG application with triggers

A weak implementation would store citations, invalidate answers when a cited document changes, and perhaps run a verifier again. That is not a strong IVM project. It lacks a nontrivial maintained query, alternative derivations, formal delta semantics, an independent correctness oracle, and a meaningful space-time trade-off.

The intended system instead maintains a standing family of queries over a shared registry of answers and claims. One passage can support many claims; one claim can have many alternative witnesses; one evidence requirement can have many witnesses; one claim can have multiple alternative conjunctive groups; and one answer aggregates multiple claims. Updates cross zero boundaries at several levels. The engine must avoid enumerating the fully expanded answer-claim-group-requirement-witness join.

This is a natural database problem because the same semantic facts are reused across many maintained outputs and because most updates are local relative to the registered state. It is a natural AI problem because the input relations that matter most--claim extraction, candidate admission, entailment, contradiction, and evidence grouping--are produced by learned models and must be evaluated for quality and cost.

3. Design principles

3.1 Exactness begins after semantic materialization

Retrieval and verification are operators with empirical error. Their outputs become immutable, versioned records. Delta propagation over those records is exact. The full-recomputation oracle receives the same records; it does not silently rerun a stochastic model.

3.2 Store observations, derive decisions

The verifier produces a score vector, not a permanent truth label. A versioned decision policy converts scores to SUPPORT, REFUTE, or NEUTRAL. Changing thresholds, calibration, or source-trust policy creates ordinary relational deltas and does not require another verifier call.

3.3 History is immutable; activity is temporal

Document versions, chunk versions, model runs, prompts, scores, and events are append-only. Active state is represented by validity intervals or epoch-indexed activation, never by overwriting historical content.

3.4 The derivation graph is bounded and acyclic

The FYP supports direct witnesses and depth-bounded evidence groups. It does not implement recursive reasoning, arbitrary Datalog, or self-referential agent memory.

3.5 Pending semantic work is visible state

An epoch can be structurally current but semantically incomplete. The API must never label an answer as confirmed for epoch e if relevant neural jobs for e remain pending. Grounding state and evaluation completeness are separate dimensions.

3.6 Exact invalidation, measured discovery

Withdrawing an active passage can find all stored dependants through reverse indexes. Discovering that a new passage affects an old uncited claim requires approximate semantic search. The architecture must expose and evaluate this asymmetry rather than hide it.

3.7 Factorize repeated structure

Maintain counts and projections along a view tree. Do not materialize every combination of answer, claim, group, requirement, and witness. Provenance is a compact certificate or token set, not an eagerly expanded polynomial.

4. State-of-the-art IVM ideas and their precise role

4.1 Signed delta streams from DBSP

DBSP formalizes incremental computation over streams using differentiation and integration and supports relational operations, aggregation, and insert/delete changes [1]. GroundLoop should adopt its clean logical model: each epoch emits a finite Z-set delta, where insertion has positive multiplicity and withdrawal has negative multiplicity. This is a semantic model, not a decision to build the FYP on Feldera.

For a batch update to relations R and S:

```
delta(R join S)
    = deltaR join S
    + R join deltaS
    + deltaR join deltaS
```

The third term matters when replacement withdraws old tuples and inserts new tuples in one logical epoch. The implementation may use before/after snapshots to avoid mistakes, but the declared semantics must match a signed batch.

4.2 Factorized higher-order views from F-IVM

F-IVM maintains a hierarchy of simpler keyed views with task-specific ring payloads and factorizes keys, payloads, and updates [2]. GroundLoop adopts two ideas:

- the evidence hierarchy is a tree of keyed count views rather than one large

materialized join;

- the same structural keys can carry multiple payloads, initially counts,

extrema, and compact provenance certificates.

GroundLoop does not initially implement arbitrary ring-generic code. It first implements explicit integer counts and tested payload functions. A generic payload interface is justified only after two payload families are working.

4.3 Join-free propagation intuition from CROWN

CROWN shows that dynamic conjunctive-query maintenance can avoid expensive materialized join propagation by using semijoin and projection structures for appropriate query classes [3]. GroundLoop's bounded hierarchy has the same physical lesson: a witness delta should update the relevant requirement count, then only propagate when satisfaction crosses zero; a requirement transition updates its group; a group transition updates its claim; and so on. The engine does not enumerate all surviving witnesses after every update.

This is inspiration, not a claim that GroundLoop's neural workload is a CROWN query class or that Secure CROWN code should be copied.

4.4 Heavy-light partitioning

Recent fully dynamic IVM work combines delta queries, trees of materialized views, and heavy-light partitioning of join keys [4]. GroundLoop has naturally skewed fanout: a policy page or popular passage may affect thousands of claims, whereas most passages affect few. A later optimized engine should therefore consider two paths:

- light key: traverse direct reverse dependencies or candidate edges;
- heavy key: use preaggregated claim-cluster summaries, batched verifier

calls, and dedicated materialized projections.

The partition threshold should be chosen from measured maintenance cost and rebalanced periodically. It is not a magic fixed degree. This optimization is an M4/M5 experiment, not part of the trusted M1 oracle.

4.5 Semiring-aware maintenance

Provenance semirings show how relational derivations can carry annotations such as why-provenance [5]. Very recent theory also shows that IVM complexity can fundamentally depend on the chosen semiring [6]. GroundLoop should take this warning seriously: counts, Boolean existence, maxima, confidence, and provenance do not automatically obey the same algebra.

The core engine therefore uses signed integer multiplicities for fully dynamic insert/delete maintenance. It derives Boolean existence from count > 0 and maintains maxima with a multiset or heap plus reference counts. Confidence is not casually called a semiring. A probabilistic payload is a later research extension requiring explicit independence or dependence assumptions.

The 2026 semiring dichotomy concerns insert-only maintenance over semirings without additive inverses [6]. GroundLoop's deletion-heavy workload cannot quote its constant-time results as a direct guarantee.

4.6 Cost-based refresh planning

Enzyme chooses refresh strategies for pipelines using a cost-based optimizer and exploits batching [7]. GroundLoop should similarly choose among:

```
direct incremental propagation
selective semantic re-verification
batched re-verification of a high-fanout region
full semantic refresh of a claim partition
```

The choice depends on candidate count, model-call cost, update fanout, stored state, and allowed risk. “Always incremental” is not a sound optimizer.

4.7 SQL-generated IVM and general engines

OpenIVM compiles view maintenance into SQL and can combine PostgreSQL with DuckDB [8]. Feldera compiles standing SQL views to DBSP circuits. These make good implementation references and possible baselines. They do not remove the GroundLoop research problem because they cannot decide which unobserved claim-passage semantic pairs need an expensive neural evaluation.

4.8 Semantic operator optimization

Recent semantic query engines treat model calls as first-class expensive operators. Sema uses runtime reordering, fusion, and prompt batching [9]; Larch learns selectivity or evaluation order for semantic predicates [10]; SPEAR versions prompts and applies adaptive refinement policies [11]. Therefore, “batch LLM calls” or “learn which neural predicate to execute first” is not a credible novelty claim by itself.

GroundLoop's stronger opportunity is temporal: maintain the consequences of previous semantic calls, determine which missing calls could change registered answers after a data update, and schedule those calls under an explicit stale risk budget.

4.9 Streaming vector maintenance is an enabler, not novelty

VectraFlow already studies streaming vector filters, top-k, and joins [12]. GroundLoop can use a vector index for reverse candidate discovery, but should not present “incremental vector retrieval” as its principal contribution.

5. Related RAG and provenance work

FreshCache models stale cache risk and selectively reuses open-web RAG results [13]. GroundLoop differs by maintaining claim-level alternative and conjunctive evidence dependencies, but FreshCache is a serious baseline for risk-aware refresh.

ProvenanceGuard performs atomic claim verification with explicit source attribution [14], and GenProve produces typed, fine-grained generation-time provenance [15]. These systems can inform how GroundLoop acquires its initial claim-source relations. GroundLoop's distinct question is how those relations and their consequences are maintained after the source collection changes.

Classical provenance work and dynamic knowledge-graph provenance demonstrate that alternative derivations can be maintained over structured data. HUKA, for example, maintains provenance polynomials for standing queries over a changing knowledge graph [17]. Minimal Evidence Groups formalize alternative minimal sets of evidence that collectively support a claim [18]. GroundLoop uses a bounded, operational form of this structure, but its hard boundary is that text-to-claim and claim-to-evidence edges are learned, versioned observations rather than assumed structured facts.

MemoRepair is an especially close conceptual warning: it withdraws invalidated derived agent-memory descendants before validated successors are republished [19]. GroundLoop should adopt the same conservative publication instinct, but the problems are not identical. MemoRepair assumes complete influence provenance for a general derived-artifact graph and optimizes cascade repair; GroundLoop studies old RAG claims, approximate discovery of previously unknown new evidence, bounded evidence alternatives, and exact factorized state maintenance. MemoRepair must appear in the final related-work comparison, not be dismissed as generic memory work.

HoH supplies a dynamic benchmark showing that outdated information can harm both retrieval and generation in RAG [20]. It is a candidate workload source, not a maintenance algorithm.

The defensible positioning is therefore:

> *GroundLoop studies versioned neural predicates as selectively acquired delta relations and exact factorized maintenance of their consequences for old RAG answers.*

No first or state-of-the-art claim is justified at design time.

6. Concepts and identifiers

The word “passage” is overloaded in the earlier design. Use the following identities:

- DocumentId: stable logical source across versions.
- DocumentVersionId: immutable content revision.
- ChunkVersionId: immutable chunk derived from exactly one document version

under a particular chunking configuration.

- ContentHash: hash of normalized chunk content; equal hashes permit exact

content reuse but do not imply equal location or authority.

- ClaimId: stable atomic claim belonging to an immutable answer version.
- SemanticObservationId: immutable model execution result for an exact input

bundle.

- DecisionId: derived label for an observation under a policy version.
- EpochId: total order for corpus and policy events.

Do not preserve ChunkVersionId across rechunking. Optional lineage is stored separately:

```
ChunkLineage(old_chunk, new_chunk, relation, overlap_score, method_version)
```

relation may be EXACT_CONTENT, SPLIT, MERGE, EDITED, or UNRELATED. Only EXACT_CONTENT can reuse a semantic observation without another semantic assumption. Reuse across an edit is a scheduler decision and must remain auditable.

7. Logical data model

The schema below is logical. Physical tables may normalize common version fields.

7.1 Corpus and event relations

```
Document(document_id, source_uri, authority_class)

DocumentVersion(
    document_version_id, document_id, content_hash,
    created_epoch, valid_from_epoch, valid_to_epoch
)

ChunkVersion(
    chunk_version_id, document_version_id, chunk_index,
    text_hash, chunker_version, text,
    valid_from_epoch, valid_to_epoch
)

CorpusEvent(
    event_id, epoch_id, event_type, payload_hash,
    recorded_at, status
)

ChunkLineage(
    old_chunk_version_id, new_chunk_version_id,
    relation, overlap_score, method_version
)
```

Validity intervals are half-open: [valid_from_epoch, valid_to_epoch). A null upper bound means active. The initial implementation may maintain an explicit active index, but its semantics must equal interval evaluation.

7.2 Registered answer relations

```
Question(question_id, text, created_epoch)

AnswerVersion(
    answer_version_id, question_id, text,
    generator_model_version, prompt_version, created_epoch
)

Claim(
    claim_id, answer_version_id, text,
    extractor_model_version, extractor_prompt_version,
    importance_weight, required
)
```

Answers and claims are immutable. Regeneration creates a new answer version; it does not rewrite the old answer whose history is being audited.

7.3 Candidate and neural observation relations

```
CandidateEvidence(
    candidate_id, claim_id, chunk_version_id,
    discovery_method_version, score, rank,
    admitted_epoch, retired_epoch
)

SemanticObservation(
    observation_id, claim_id, chunk_version_id,
    task_type, support_score, refute_score, neutral_score,
    model_id, model_version, prompt_version,
    decoding_config_hash, input_hash,
    produced_epoch, run_seed, raw_output_hash
)

DecisionPolicy(
    policy_version, support_threshold, refute_threshold,
    margin_rule, calibration_version, source_policy_version,
    valid_from_epoch, valid_to_epoch
)

ObservationDecision(
    observation_id, policy_version, label,
    calibrated_support, calibrated_refute
)
```

CandidateEvidence says a pair is worth considering. It is not evidence and does not affect grounding. SemanticObservation records model output. ObservationDecision is a deterministic view of scores and policy.

If a model does not expose meaningful three-way probabilities, store its raw logits or task-specific scores and document the calibration mapping. Do not fabricate normalized probabilities.

7.4 Bounded evidence relations

```
EvidenceGroup(group_id, claim_id, group_type, group_model_version)

EvidenceRequirement(
    requirement_id, group_id, requirement_text,
    requirement_model_version
)

RequirementWitness(
    requirement_id, observation_id, witness_policy_version
)
```

Core group semantics are monotone Boolean structure over currently active decisions:

- witnesses within a requirement are alternatives (OR);
- requirements within a group are conjunctive (AND);
- groups for a claim are alternatives (OR).

Direct support is represented as a distinguished single-requirement group or as a separate direct-support view. For implementation clarity, M1 can use a direct view and M5 can normalize both forms to the same group algebra.

Refutation is direct at first. Conjunctive refutation groups are excluded from the FYP unless a dataset and annotation policy justify them.

7.5 Job and publication relations

```
SemanticJob(
    job_id, epoch_id, claim_id, chunk_version_id,
    job_type, priority, estimated_cost, estimated_risk_reduction,
    status, attempt, created_at, completed_at
)

Epoch(
    epoch_id, event_id, structural_status, semantic_status,
    opened_at, sealed_at, publication_mode
)

PublishedAnswerState(
    answer_version_id, published_epoch,
    grounding_state, evaluation_state,
    lower_supported_claims, upper_supported_claims,
    refuted_claims, conflicted_claims
)
```

8. Canonical maintained views

8.1 Active observations and decisions

For epoch e:

```
ActiveChunk_e(p) := ChunkVersion(p) and e in validity_interval(p)

ActiveDecision_e(o, c, p, label) :=
    SemanticObservation(o, c, p, ...)
    join ActiveChunk_e(p)
    join CurrentDecisionPolicy_e(k)
    join ObservationDecision(o, k, label)
```

An observation is not deleted when its chunk becomes inactive. It simply leaves ActiveDecision_e.

8.2 Direct counts

```
DirectSupportCount(c) := count of distinct active observation IDs
                         with label SUPPORT for c

DirectRefuteCount(c)  := count of distinct active observation IDs
                         with label REFUTE for c
```

The key is observation identity, not the raw model output text. Exactly replayed observations are deduplicated by input_hash plus model execution identity according to an explicit idempotence rule.

8.3 Requirement and group views

```
RequirementWitnessCount(r) := number of active SUPPORT observations
                              linked to requirement r

RequirementSatisfied(r) := RequirementWitnessCount(r) > 0

GroupRequirementCount(g) := number of declared requirements in g

GroupSatisfiedCount(g) := number of requirements r in g
                          for which RequirementSatisfied(r)

GroupComplete(g) := GroupRequirementCount(g) > 0 and
                    GroupSatisfiedCount(g) = GroupRequirementCount(g)

CompleteGroupCount(c) := number of complete groups belonging to c
```

Empty groups are invalid rather than vacuously complete.

8.4 Claim state

```
supported(c) := DirectSupportCount(c) > 0
                or CompleteGroupCount(c) > 0

refuted(c) := DirectRefuteCount(c) > 0

supported and refuted      -> CONFLICTED
supported and not refuted  -> SUPPORTED
not supported and refuted  -> REFUTED
otherwise                  -> UNSUPPORTED
```

Canonical state stores counts, best scores, witness certificate IDs, and the label. The label alone is insufficient.

8.5 Answer state

For required claims only:

```
any REFUTED claim                          -> CONTRADICTED
else any CONFLICTED claim                  -> CONFLICTED
else all required claims SUPPORTED         -> VALID
else at least one required claim SUPPORTED -> PARTIALLY_SUPPORTED
else                                       -> UNSUPPORTED
```

An answer with no required claims is invalid input. Optional claims can be displayed but do not silently make an answer valid.

REGENERATION_RECOMMENDED is derived by a separate action policy using grounding state, claim importance, age, user settings, and regeneration cost. It must never replace the canonical grounding label.

9. Pending-work bounds

GroundLoop cannot make insertion discovery and neural verification atomic with a cheap corpus write. It therefore maintains two state dimensions:

```
GroundingState  = SUPPORTED | UNSUPPORTED | REFUTED | CONFLICTED
EvaluationState = COMPLETE | PENDING | DEGRADED | FAILED
```

For each claim at epoch e:

```
known_support(c)  = completed active support witnesses
pending_support(c)= admitted pairs whose semantic jobs could add support

support_lower(c) = indicator(known_support(c) > 0)
support_upper(c) = indicator(known_support(c) + pending_support(c) > 0)
```

Equivalent bounds are kept for refutation. These are logical possibility bounds, not calibrated probabilities. They permit useful statements:

- lower = upper = 1: support is established despite pending work;
- lower = 0, upper = 1: support is unresolved;
- lower = upper = 0: no admitted support path exists, but this is only complete

if candidate discovery is sealed under the declared policy.

The API should return both the last complete state and the provisional state:

```
confirmed_as_of_epoch = 41: VALID
latest_epoch = 42: PENDING, possible range VALID..CONFLICTED
```

This avoids falsely calling old state current and provides a nontrivial incremental view while semantic work is asynchronous.

10. Factorized physical view tree

The logical dependency path is:

```
ChunkVersion
  -> ObservationDecision
    -> RequirementWitnessCount
      -> RequirementSatisfied
        -> GroupSatisfiedCount
          -> GroupComplete
            -> ClaimState
              -> AnswerState
```

Store keyed maps or indexed tables at each aggregation boundary:

```
observations_by_chunk[chunk]
requirements_by_observation[observation]
group_by_requirement[requirement]
claim_by_group[group]
answer_by_claim[claim]
```

A witness insertion changes one requirement count. If the old count was positive, RequirementSatisfied does not change, so propagation stops. If it crosses 0 -> 1, one group satisfied count changes. Propagation again stops unless the group crosses incomplete to complete. The same zero-crossing logic applies in reverse for deletion.

This is the central IVM mechanism. It is more than cached status labels: it is a factorized hierarchy that prevents the cost of an update from depending on the number of alternative witnesses when no Boolean boundary changes.

11. Delta rules

11.1 Signed count maintenance

For a witness delta (r, weight) with weight in {+1, -1}:

```
old_count = RequirementWitnessCount[r]
new_count = old_count + weight
assert new_count >= 0

if (old_count == 0) != (new_count == 0):
    satisfaction_delta = +1 if new_count > 0 else -1
    propagate_to_group(r, satisfaction_delta)
```

The group handler is analogous:

```
old_complete = satisfied_count == requirement_count
new_satisfied_count = satisfied_count + satisfaction_delta
new_complete = new_satisfied_count == requirement_count

if old_complete != new_complete:
    propagate_to_claim(group, +1 if new_complete else -1)
```

Claim and answer labels are recomputed only for keys reached by these deltas.

11.2 Max-score maintenance

Deletion makes a scalar maximum non-invertible. Do not update best_support_score by subtraction. Maintain a counted multiset of active scores per claim, for example an ordered index or heap with lazy deletion. The maximum is the greatest score with positive multiplicity.

11.3 Policy delta

When policy k1 is replaced by k2:

1. identify observations whose decisions can change;
2. evaluate deterministic decisions for those scores under k2;
3. compute Decision(k2) - Decision(k1) as signed label deltas;
4. propagate only observations whose label or calibrated value changed;
5. publish states at the new policy epoch.

For a pure threshold change, maintain score indexes and range-scan only the interval crossed by the old and new thresholds. For a source-trust change, restrict work to observations from affected sources. An arbitrary new calibration function can change every decision and may require a full decision refresh; the cost-based planner must admit that rather than calling it a cheap delta. Piecewise-monotone calibration permits interval indexing only when its breakpoints and ordering guarantees are explicit.

This is a genuine neural-computation saving: thresholds or calibration can be changed without rerunning the verifier. It also creates a useful experiment on view-definition evolution.

11.4 Replacement delta

A document replacement is a single epoch containing:

- negative activity deltas for old chunk versions;
- positive activity deltas for new chunk versions;
- optional exact-reuse deltas for content-identical chunks;
- new candidate and semantic-job deltas.

No intermediate state between deletion and insertion is externally published. Provisional state may be visible only with EvaluationState=PENDING.

12. Bidirectional semantic delta join

The learned relation SemanticallyRelevant(claim, chunk) is not fully materialized. GroundLoop maintains only selected candidate pairs. Updates use two different directions.

12.1 Withdrawal path: chunk to known claims

For a deleted or deactivated chunk:

1. read observations_by_chunk and candidates_by_chunk;
2. withdraw active decisions exactly;
3. propagate count and state deltas;
4. identify claims that lost their last support route;
5. request replacement candidates only for those claims or for policy-defined

high-risk claims.

Relative to stored dependencies, this affected set is exact.

12.2 Admission path: new chunk to registered claims

For an inserted chunk:

1. embed or otherwise index the chunk;
2. query a reverse claim index for top L candidate claims;
3. add lexical/entity candidates to protect against embedding misses;
4. apply cheap deterministic filters;
5. rank candidate pairs by predicted state impact;
6. invoke the verifier only for admitted pairs;
7. insert immutable observations and propagate their deltas.

This path is approximate. Its primary metric is affected-claim recall against a full candidate-and-verification audit, not ordinary retrieval precision alone.

12.3 Candidate frontier

For each claim maintain:

- active verified witnesses;
- an admitted verification frontier;
- a reserve top-k candidate frontier;
- a lower-scoring tail represented only in the retrieval index.

When a witness disappears, promote from reserve before launching a broad search. This is analogous to maintaining extra order-statistic state for top-k views. Frontier size is a tunable space/neural-cost parameter.

12.4 Why this could be a research contribution

The database structure does not make semantic discovery exact. The contribution is the explicit composition of:

- exact reverse provenance for withdrawals;
- reverse semantic retrieval for insertions;
- incremental top-k frontier repair;
- exact downstream factorized propagation;
- measured end-to-end recall and cost.

Call this mechanism Bidirectional Semantic Delta Join (BSDJ) as a working name. It is a design label, not a literature priority claim.

13. Risk-bounded neural IVM

Neural calls dominate cost and candidate discovery can miss affected claims. GroundLoop should make this trade-off explicit.

For candidate job j, estimate:

```
p_change(j)   probability the observation changes a claim-relevant count
impact(j)     importance-weighted loss if that change remains unknown
cost(j)       predicted tokens, latency, or monetary cost
uncertainty(j)model/calibration uncertainty

value(j) = p_change(j) * impact(j) * uncertainty(j) / cost(j)
```

Under budget B, schedule jobs to maximize expected risk reduction, subject to safety constraints:

- always verify candidates for critical claims above a risk floor;
- cap time since last full audit;
- reserve budget for exploration, not only high predicted value;
- do not mark an epoch complete until its declared candidate policy is

exhausted or a documented risk threshold is accepted.

This becomes Risk-Bounded Neural IVM (RB-NIVM). The exact engine remains exact over completed observations. The scheduler's safety is empirical and is reported as stale-answer risk, affected-claim recall, and time-to-detection.

The simplest FYP version uses a transparent logistic model or calibrated rules. A reinforcement-learning scheduler is unjustified unless simpler policies are already measured and a sufficiently large workload exists.

14. Heavy-light semantic fanout

Define fanout statistics:

```
dep_degree(p)       number of active claims with stored dependency on chunk p
candidate_degree(p) number of registered claims selected for new chunk p
claim_degree(c)     number of active or reserve evidence candidates for claim c
```

Choose threshold tau from measured costs. For light keys, direct indexed propagation is efficient. For heavy keys, use:

- batch withdrawal or admission;
- vectorized verifier prompts where model semantics permit batching;
- preaggregated claim partitions;
- one scan of a dense bitmap or sorted claim-ID vector;
- deferred micro-batches to amortize model and transaction overhead.

Reclassification requires a major/minor rebalancing policy. A key does not switch partitions on every small degree fluctuation; use hysteresis around tau and record rebalancing cost.

This is valuable only if the benchmark contains genuine skew. If fanout is uniform and small, heavy-light machinery should be reported as unnecessary.

15. Provenance payload and explanation certificates

Full provenance polynomials can grow very large. GroundLoop needs explanations, not complete symbolic expansion on every update. Maintain:

- exact integer multiplicity;

- one or a small bounded set of current witness IDs;
- one current complete-group certificate per supported claim;
- a refutation certificate when present;
- the event and epoch that caused a boundary crossing.

A certificate for a complete group contains one active witness per requirement. If the selected witness is deleted but alternatives remain, replace the certificate locally without changing group completeness. This produces a compact why explanation while counts preserve exact state.

Optional audit mode can enumerate all witnesses or construct a provenance polynomial on demand from base relations. Do not place the entire polynomial in the hot maintenance path.

16. Epoch and consistency protocol

16.1 State machine

```
RECEIVED
  -> STRUCTURAL_COMMITTED
  -> SEMANTIC_PENDING
  -> SEMANTIC_COMPLETE
  -> SEALED
```

Exceptional states are DEGRADED and FAILED. A retry does not create a new semantic observation unless it completes with a distinct execution identity.

16.2 Processing protocol

1. Assign a monotonically ordered epoch and validate event idempotence.
2. In a short transaction, append new versions and close old validity

intervals.

3. Apply exact withdrawal deltas and create candidate-discovery jobs.
4. Commit structural state with SEMANTIC_PENDING.
5. Execute model work outside the transaction.
6. Append each completed immutable observation and incrementally maintain

provisional views in idempotent microtransactions.

7. When the epoch's declared scheduling policy is satisfied, run invariant and

optional differential checks.

8. Atomically publish the sealed answer-state snapshot and its delta log.

16.3 Publication modes

- Strict: serve the last sealed epoch until the new epoch seals.
- Provisional: serve the latest state with bounds and PENDING status.

The recommended API supports both. The dashboard defaults to provisional for research visibility but clearly displays confirmed_as_of_epoch.

16.4 Idempotence

Use unique constraints on event ID, job ID, observation execution key, and delta application key. Each delta application records (epoch, source_record, view_name, key, weight). A replay is a no-op; the same identifier with a different payload hash is a conflict.

17. Reference and optimized engines

GroundLoop requires three conceptually distinct paths.

17.1 Pure reference recomputation

Given a snapshot of base relations, recompute all active decisions, counts, group completeness, claim states, and answer states from scratch. This path is simple, slow, and independent of incremental mutation logic.

17.2 GroundLoop incremental engine

Consume signed base deltas, maintain factorized keyed views, stop propagation at unchanged zero boundaries, and emit status deltas and certificates.

17.3 Full semantic refresh baseline

For each relevant registered claim, rerun retrieval and verification under a fixed model/prompt configuration. This is an empirical oracle, not the relational correctness oracle. Model nondeterminism must be controlled or measured.

Using the incremental engine to implement the reference path would make the differential test circular. Do not do it.

18. Implementation-platform decision

Option A: custom Python only

Advantages: fastest route to transparent semantics and tests. Disadvantages: weak scalability and limited database credibility if it remains the final system.

Option B: PostgreSQL triggers and materialized tables

Advantages: durable, inspectable, familiar. Disadvantages: complex trigger ordering, awkward async neural calls, and risk that the project looks like application plumbing rather than an IVM design.

Option C: Feldera/DBSP for all structured views

Advantages: principled signed deltas and rich SQL IVM. Disadvantages: an external engine can obscure what the student designed, and it does not solve semantic candidate acquisition or transaction/publication integration.

Option D: layered implementation (recommended)

1. dependency-light Python reference semantics;
2. PostgreSQL durable base/event/observation store;
3. a custom GroundLoop factorized maintenance module with explicit delta

rules;

4. SQL full-recompute queries as a second oracle;
5. optional Feldera/OpenIVM comparison for selected structured views.

This makes the IVM contribution visible while retaining credible database engineering. A C++ kernel is unnecessary unless profiling proves Python is the bottleneck after model calls are excluded.

19. Core algorithms

19.1 Apply a completed observation

```
apply_observation(epoch, observation):
    assert immutable inputs and active referenced chunk
    append observation idempotently
    decision = evaluate_policy(observation.scores, current_policy(epoch))
    append decision

    if decision.label == SUPPORT:
        for requirement in requirements_by_observation[observation.id]:
            apply_requirement_witness_delta(epoch, requirement, +1)
        apply_direct_support_delta_if_applicable(epoch, claim, +1)

    if decision.label == REFUTE:
        apply_direct_refute_delta(epoch, claim, +1)

    update_pending_job_count(epoch, claim, -1)
    update_claim_bounds(claim)
```

19.2 Withdraw a chunk

```
withdraw_chunk(epoch, chunk):
    close chunk validity interval
    affected_claims = empty set

    for observation in active_observations_by_chunk[chunk]:
        decision = current_decision[observation]
        propagate inverse delta for decision
        affected_claims.add(observation.claim)

    for claim in affected_claims:
        if claim lost its final support route or policy requires refresh:
            enqueue frontier repair(claim)
```

The loop is proportional to known dependencies. In the optimized engine, heavy chunks can follow a dense/batched path.

19.3 Admit a new chunk

```
admit_chunk(epoch, chunk, budget):
    candidates = union(
        reverse_vector_topL(chunk, registered_claims),
        lexical_entity_candidates(chunk),
        lineage_candidates(chunk)
    )
    deduplicate candidates
    features = estimate state impact and cost
    selected = risk_bounded_scheduler(candidates, budget)

    for pair in selected:
        append CandidateEvidence
        enqueue SemanticJob

    update claim pending bounds
```

19.4 Seal an epoch

```
seal(epoch):
    require no required jobs pending
    require all signed counts nonnegative
    require group satisfied_count <= requirement_count
    compare incremental and full relational state in test/audit mode
    atomically publish changed answer states and certificates
    mark epoch SEALED
```

20. Correctness contract and invariants

20.1 Relational equivalence

After every sealed epoch:

```
IncrementalView(base_snapshot_e) == FullRecompute(base_snapshot_e)
```

Equality includes counts, Boolean boundaries, best scores, canonical labels, and certificate validity. A certificate need not be identical to the oracle's chosen certificate when multiple witnesses exist, but it must be valid.

20.2 Invariants

1. Historical content and observations are immutable.
2. At most one active document version exists per logical document.
3. Every active chunk belongs to an active document version.
4. Every observation refers to an existing claim and immutable chunk version.
5. A decision refers to exactly one observation and policy version.
6. Maintained multiplicities never become negative.
7. Satisfied group requirements never exceed declared requirements.
8. Empty groups are invalid.
9. COMPLETE implies no required semantic job remains under the epoch policy.

10. Every published state delta has an event, epoch, old state, new state, and a valid affected-key certificate.
11. Event replay cannot duplicate versions, jobs, observations, or view deltas.
12. Old versions remain auditable after deactivation.

20.3 Non-guarantees

The contract does not prove:

- claim extraction correctness;
- candidate-discovery completeness;
- verifier truthfulness;
- independence of confidence scores;
- correctness outside the active corpus and model policy;
- recursive reasoning soundness;
- security or privacy.

21. Experimental design

21.1 Systems hypotheses

- S1: factorized delta propagation equals full relational recomputation after every event.
- S2: zero-crossing propagation touches fewer hierarchy nodes than eager recomputation when alternative witnesses are common.
- S3: candidate frontiers reduce replacement verifier calls without a material loss in affected-claim recall.
- S4: cost-based selection correctly chooses full refresh for sufficiently high-fanout updates.
- S5: heavy-light execution improves tail latency only on skewed workloads.

21.2 AI hypotheses

- A1: combined reverse-vector, lexical/entity, and lineage discovery has higher affected-claim recall than embedding-only discovery at the same verifier budget.
- A2: risk-bounded scheduling reduces stale-answer exposure for a fixed call budget relative to similarity ranking.
- A3: score calibration improves action-policy decisions but does not alter the exact relational equivalence claim.
- A4: bounded evidence groups reduce false invalidation relative to direct citation invalidation.

21.3 Workload axes

Vary independently:

- registered answers and claims;
- active chunks;
- direct witness alternatives;
- requirements per group and groups per claim;
- update locality;
- source/claim fanout skew;
- insert/delete/replace mixture;
- semantic candidate budget;
- model latency and batch size;
- proportion of changed chunks that are semantically answer-affecting.

21.4 Required baselines

1. full retrieval and verification refresh;
2. full answer regeneration where affordable;
3. TTL or freshness-risk refresh, including a FreshCache-style policy;
4. source-level invalidation;
5. direct-citation invalidation;
6. GroundLoop direct witnesses without groups;
7. GroundLoop groups without risk scheduling;
8. GroundLoop full design;
9. optional general IVM engine for structured views.

21.5 Metrics

Systems:

- median, p95, and p99 update latency;
- structured tuples and view keys touched;
- verifier calls, generated tokens, and batches;
- candidate-discovery work;
- maintained state size;
- full-refresh speedup and break-even point;
- provisional-to-sealed time;
- rebalancing and audit costs.

AI and safety:

- affected-claim recall and precision;
- stale-answer exposure time;
- support/refute/neutral macro-F1;
- calibration error and Brier score where probabilities are valid;

- false invalidation and missed invalidation;
- answer-state agreement with full semantic refresh;
- risk per unit cost and area under risk-versus-budget curve.

21.6 Correctness testing

- table-driven truth tests for every state combination;
- property-based generated base snapshots;
- randomized signed insert/delete/replace streams;
- differential comparison after every micro-batch and sealed epoch;
- event replay and conflicting-payload tests;
- failure injection between structural commit, observation commit, and seal;
- policy-threshold update streams;
- certificate validity checks with alternative witnesses.

Neural metrics must never be merged into the deterministic equality test.

22. Design decisions to approve

The following are the recommended decisions for design freeze.

Decision	Recommendation	Rejected alternative	Reason
Semantic record	immutable raw scores plus versioned derived decision	permanent verifier label	permits policy deltas and honest calibration
Chunk identity	immutable per document version; separate lineage	stable ID reused across edits	avoids invalid provenance after rechunking
Dynamic algebra	signed integer multiplicities	Boolean flags only	deletion, duplicates, and zero crossings are explicit
Evidence structure	bounded OR-AND-OR DAG	arbitrary recursive graph	meaningful IVM with feasible extraction/evaluation
Consistency	epochs plus COMPLETE/PENDING/DEGRADED	overwrite current state after each model call	prevents false freshness and partial publication
Insert discovery	reverse claim retrieval plus lexical/entity union	only recheck cited claims	new evidence can affect uncited old claims
Delete discovery	exact reverse provenance	semantic search after deletion	known dependencies should not be rediscovered approximately
Provenance	counts plus bounded certificates	full expanded provenance in hot path	exact state with controlled space
Answer action	separate regeneration policy	REGENERATION_RECOMMENDED as evidence label	action and evidence are different semantics
Engines	Python oracle + PostgreSQL + custom IVM	triggers only or immediate C++	strongest correctness/portfolio/feasibility balance
First novel extension	pending bounds + policy-delta maintenance	probabilistic semiring immediately	more defensible and implementable
Advanced extension	risk-bounded scheduler, then heavy-light fanout	RL scheduler first	measurable with simpler models and lower risk

Two decisions deserve explicit discussion before coding beyond M1:

1. Should direct support be a separate fast path or normalized immediately as a one-requirement evidence group? Recommendation: separate in M1, unified logical semantics by M5.

2. What seals an epoch: exhausting a fixed top-L candidate policy, reaching a calibrated risk bound, or a user latency deadline? Recommendation: fixed top-L for the first reproducible system; risk-bound sealing as an evaluated extension.

23. Revised milestone plan

M0.5 -- design freeze

- review this document;
- resolve Section 22 decisions;
- freeze canonical schemas, state truth tables, epoch semantics, and exact claims;
- update the implementation prompt.

M1 -- deterministic reference semantics

- immutable versions and validity intervals;
- raw semantic score observations;
- versioned decision policy;
- pure full recomputation;
- direct support/refute counts;
- claim and answer state;
- event idempotence and audit history;
- one replacement scenario and one policy-change scenario.

No optimized IVM or neural model is used yet.

M2 -- structured delta engine

- signed delta records;
- factorized direct-witness views;
- PostgreSQL base store;
- zero-crossing propagation;
- differential tests after every event;
- provisional and sealed epoch state.

M3 -- static neural pipeline

- versioned chunking and embeddings;
- claim extraction;
- score-producing verifier and calibration;
- candidate frontier;
- end-to-end static answer registration.

M4 -- bidirectional semantic delta join

- exact withdrawal path;
- reverse insertion discovery;
- lexical/entity union;
- frontier repair;
- full semantic refresh baseline;
- affected-claim recall and call-savings evaluation.

M5 -- evidence groups and compact certificates

- bounded OR-AND-OR schema;
- group factorized IVM;
- controlled/gold evaluation before model-proposed groups;
- alternative-evidence false-invalidation study.

M6 -- risk and physical optimization

- risk-bounded scheduler;
- cost-based incremental-versus-refresh decision;
- optional heavy-light fanout partitioning;
- policy-delta experiments;
- full application and dissertation experiments.

If time is limited, heavy-light partitioning is the first feature to drop. The exact delta engine, semantic impact evaluation, and complete demo are more important for an FYP.

24. Novelty assessment

High-confidence statements

- The exact/empirical boundary is technically coherent.
- Factorized IVM over stored neural observations is a legitimate AI-database intersection.
- Policy-delta maintenance and pending-work bounds are implementable within an FYP.
- Exact deletion plus approximate insertion discovery is the correct systems model for this problem.

Moderate-confidence research hypotheses

- BSDJ is a useful unifying abstraction for dynamic claim-evidence maintenance.
- RB-NIVM can produce a better risk-cost frontier than similarity-only scheduling.
- Heavy-light treatment of semantic fanout can improve tail performance on

shared-source workloads.

Low/unknown-confidence novelty claims

- No existing system combines all proposed elements.
- The proposed names or algorithms are publication-level novel.
- The chosen evidence-group algebra is the best semantic representation.
- A probabilistic payload will be both sound and efficient.

Those claims require a systematic literature review, implementation, and experiments. The project does not need them to be a strong FYP or portfolio artifact.

25. Immediate next action

Do not start with pgvector, an LLM API, or a dashboard. First review and freeze the design decisions. Then revise M1 so that its deterministic vertical slice contains:

1. immutable score observations;
2. a versioned label policy;
3. validity intervals and chunk identity rules;
4. grounding state separate from evaluation state;
5. pure full recomputation;
6. a replacement event;
7. a policy-change event that changes a label without a neural call;
8. exact idempotence and audit assertions.

Only after this oracle exists should the project implement signed incremental propagation. This ordering prevents the optimized engine from defining its own correctness.

References

1. Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen.

“DBSP: Automatic Incremental View Maintenance for Rich Query Languages.” PVLDB 16(7), 2023. <https://www.vldb.org/pvldb/vol16/p1601-budiu.pdf>

2. Milos Nikolic and Dan Olteanu. “Incremental View Maintenance with Triple

Lock Factorization Benefits.” SIGMOD 2018 / arXiv version. <https://arxiv.org/abs/1703.07484>

3. Qichen Wang, Xiao Hu, Binyang Dai, and Ke Yi. “Change Propagation Without

Joins.” PVLDB 16(5), 2023. <https://www.vldb.org/pvldb/vol16/p1046-hu.pdf>

4. Mahmoud Abo-Khamis, Eden Chmielewski, Andrei Draghici, Ahmet Kara, and Dan

Olteanu. “Maintaining Queries under Updates Using Heavy-Light Partitioning of the Input Relations.” PACMMOD 4(2), 2026. <https://doi.org/10.1145/3801905>

5. Todd J. Green, Grigoris Karvounarakis, and Val Tannen. “Provenance

Semirings.” PODS 2007. <https://doi.org/10.1145/1265530.1265535>

6. Eden Chmielewski, Andrei Draghici, Dan Olteanu, and Haozhe Zhang. “The Role

of Semirings in Incremental View Maintenance.” 2026. <https://arxiv.org/abs/2606.07795>

7. Ritwik Yadav et al. “Enzyme: Incremental View Maintenance for Data Engineering.” 2026. <https://arxiv.org/abs/2603.27775>

8. Ilaria Battiston, Kriti Kathuria, and Peter Boncz. “OpenIVM: a SQL-to-SQL Compiler for Incremental Computations.” 2024. <https://arxiv.org/abs/2404.16486>

9. Kangkang Qi et al. “Sema: A High-performance System for LLM-based Semantic Query Processing.” 2026. <https://arxiv.org/abs/2603.11622>

10. Fuheng Zhao et al. “Larch: Learned Query Optimization for Semantic Predicates.” 2026. <https://arxiv.org/abs/2606.07923>

11. Ugur Cetintemel et al. “Making Prompts First-Class Citizens for Adaptive LLM Pipelines.” CIDR 2026. <https://www.vldb.org/cidrdb/2026/making-prompts-first-class-citizens-for-adaptive-llm-pipelines.html>

12. Duo Lu et al. “VectraFlow: Integrating Vectors into Stream Processing.” CIDR 2025. <https://vldb.org/cidrdb/2025/vectraflow-integrating-vectors-into-stream-processing.html>

13. Muhammad Mansoor, Tahir Ahmad, and Yeo-Chan Yoon. “Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs.” 2026. <https://arxiv.org/abs/2607.04281>

14. Ander Alvarez, Santhiya Rajan, Samuel Mugel, and Roman Orus. “ProvenanceGuard: Source-Aware Factuality Verification for MCP-Based LLM Agents.” 2026. <https://arxiv.org/abs/2606.18037>

15. Jingxuan Wei et al. “GenProve: Learning to Generate Text with Fine-Grained Provenance.” ACL 2026. <https://aclanthology.org/2026.acl-long.228/>

16. Dan Olteanu. “Recent Increments in Incremental View Maintenance.” PODS 2024. <https://arxiv.org/abs/2404.17679>

17. Garima Gaur, Arnab Bhattacharya, and Srikanta Bedathur. “How and Why is An Answer (Still) Correct? Maintaining Provenance in Dynamic Knowledge Graphs.” 2020. <https://arxiv.org/abs/2007.14864>

18. Xiangci Li, Sihao Chen, Rajvi Kapadia, Jessica Ouyang, and Fan Zhang. “Minimal Evidence Group Identification for Claim Verification.” TrustNLP 2025. <https://aclanthology.org/2025.trustnlp-main.8/>

19. Yang Zhao, Chengxiao Dai, Mengying Kou, and Yue Xiu. “MEMOREPAIR: Barrier-First Cascade Repair in Agentic Memory.” 2026. <https://arxiv.org/abs/2605.07242>

20. Jie Ouyang et al. “HoH: A Dynamic Benchmark for Evaluating the Impact of Outdated Information on Retrieval-Augmented Generation.” ACL 2025. <https://aclanthology.org/2025.acl-long.301/>